Imię i nazwisko: Mateusz Portka
Data wykonania: 20 października 2026 r.
Kierunek / Przedmiot: Informatyka / Algorytmy Geometryczne
Numer ćwiczenia: Laboratorium 1

Sprawozdanie z Laboratorium 1

Podstawowe predykaty geometryczne

Badanie stabilności numerycznej testu orientacji punktu względem prostej
w arytmetyce zmiennoprzecinkowej IEEE 754

Spis treści

1. Dane techniczne i środowisko testowe 2
2. Podstawy teoretyczne 2
   2.1. Problem orientacji punktu względem prostej 2
   2.2. Porównywane metody obliczeniowe 2
   2.3. Wpływ arytmetyki IEEE 754 i rola parametru ε 2
3. Realizacja ćwiczenia i zbiory testowe 3
   3.1. Prosta testowa AB 3
   3.2. Specyfikacja zbiorów punktów 3
   3.3. Parametry eksperymentów 3
4. Wyniki i analiza eksperymentalna 4
   4.1. Wizualizacja wygenerowanych zbiorów danych 4
   4.2. Wizualizacja klasyfikacji punktów 5
   4.3. Wyniki dla zbiorów standardowych (Zbiór A i Zbiór C) 5
   4.4. Zbiór B: Fałszywie oznaczone punkty na prostej 6
   4.5. Zbiór D: Błędy klasyfikacji na punktach współliniowych (wersja kierunkowa) 7
   4.6. Porównanie efektywności czasowej (Benchmark) 8
5. Wnioski końcowe 9

1. Dane techniczne i środowisko testowe

Wszystkie procedury generowania zbiorów danych, obliczenia macierzowe, symulacje numeryczne oraz pomiary czasowe wykonano w następującym środowisku sprzętowo-programowym:

- **Platforma sprzętowa:** Apple MacBook Air (Apple M4, 10 rdzeni: 4 wydajnościowe Performance + 6 energooszczędnych Efficiency)
- **Pamięć operacyjna:** 24 GB zunifikowanej pamięci RAM (Unified Memory)
- **System operacyjny:** macOS 26.2 (kompilacja 25C56)
- **Środowisko i interpreter:** Python 3.13.2 (64-bit), środowisko wirtualne `.venv`
- **Zastosowane biblioteki zewnętrzne:**
  - `numpy` (wersja 2.5.4) — wektorowa reprezentacja punktów i macierzy oraz funkcja `np.linalg.det`
  - `pandas` (wersja 3.0.6) — agregacja statystyczna wyników w ramki danych i eksport CSV
  - `matplotlib` (wersja 3.11.2) — rendering i zapis wykresów wieloosiowych w podwyższonej rozdzielczości 400 DPI
  - `time.perf_counter` — precyzyjny licznik monotoniczny do pomiaru czasu działania procesora

2. Podstawy teoretyczne

2.1. Problem orientacji punktu względem prostej

Podstawowym predykatem geometrycznym w przestrzeni dwuwymiarowej jest określenie położenia badanego punktu $C = (x_c, y_c)$ względem zorientowanej prostej wyznaczonej przez dwa punkty bazowe $A = (x_a, y_a)$ oraz $B = (x_b, y_b)$.

O orientacji decyduje znak wartości wyznacznika D z uwzględnieniem progu tolerancji $\varepsilon \geq 0$:

$$\text{Kwalifikacja orientacji} = \begin{cases} \text{Lewo (L)} & \text{gdy } D > \varepsilon \\ \text{Prawo (R)} & \text{gdy } D < -\varepsilon \\ \text{Na prostej (ON)} & \text{gdy } |D| \leq \varepsilon \end{cases}$$

Geometrycznie oznacza to, że gdy poruszamy się wzdłuż prostej od punktu A do punktu B, dodatnia wartość wyznacznika wskazuje na położenie punktu C po naszej lewej stronie, ujemna — po prawej, natomiast wartość bliska zeru ($|D| \leq \varepsilon$) świadczy o współliniowości punktów.

2.2. Porównywane metody obliczeniowe

Do wyznaczenia wartości D zastosowano cztery procedury obliczeniowe:

1. `custom_3x3` — autorska implementacja wyznacznika macierzy 3×3 (formuła Sarrusa),
2. `custom_2x2` — autorska implementacja wyznacznika macierzy różnic współrzędnych 2×2,
3. `numpy_3x3` — biblioteczna funkcja `np.linalg.det` dla macierzy 3×3,
4. `numpy_2x2` — biblioteczna funkcja `np.linalg.det` dla macierzy różnic 2×2.

2.3. Wpływ arytmetyki IEEE 754 i rola parametru ε

W komputerowej reprezentacji liczb zmiennoprzecinkowych mantysa ma ograniczoną pojemność bitową: format `float32` zapewnia około 7 dziesiętnych cyfr znaczących, natomiast `float64` około 16 cyfr.

Gdy operujemy na współrzędnych o dużych rzędach wielkości (np. 10^{14}), odejmowanie bliskich sobie liczb prowadzi do zjawiska **eliminacji katastrofalnej**. Wiodące cyfry ulegają wyzerowaniu, a wynik zostaje zdominowany przez błędy zaokrągleń, co w formacie `float32` skutkuje powstaniem fałszywego zera maszynowego. Wprowadzenie progu tolerancji ε pozwala odróżnić rzeczywiste odchylenia geometryczne od drobnych błędów zaokrągleń komputera.

3. Realizacja ćwiczenia i zbiory testowe

3.1. Prosta testowa AB

Prosta bazowa została zdefiniowana przez dwa punkty:

$$A = (-1.0, \quad 0.0), \quad B = (1.0, \quad 0.1)$$

Jej postać kierunkowa przyjmuje postać:

$$y = 0.05x + 0.05$$

3.2. Specyfikacja zbiorów punktów

W ramach laboratorium wygenerowano pięć niezależnych zbiorów punktów:

- **Zestaw A (10^5 punktów):** Rozkład jednostajny w obszarze $[-1000, 1000]^2$.
- **Zestaw B (10^5 punktów):** Rozkład jednostajny w obszarze $\left[-10^{14}, 10^{14}\right]^2$.
- **Zestaw C (10^3 punktów):** Punkty na okręgu o środku $S = (0, 0)$ i promieniu $R = 100$, wyznaczane równaniem $x = R\cos\theta, y = R\sin\theta$ dla $\theta \in [0, 2\pi)$.
- **Zestaw D — wersja kierunkowa (10^3 punktów):** Punkty leżące matematycznie na prostej, generowane wzorem $y = 0.05x + 0.05$ dla $x \in [-1000, 1000]$.
- **Zestaw D — wersja parametryczna (10^3 punktów):** Punkty generowane równaniem parametrycznym dla $t \in [-500, 500]$, wyznaczającym współrzędne: $x = -1 + 2t$ oraz $y = 0.1t$.

3.3. Parametry eksperymentów

Każdy zestaw punktów przebadano dla pełnego iloczynu kartezjańskiego:

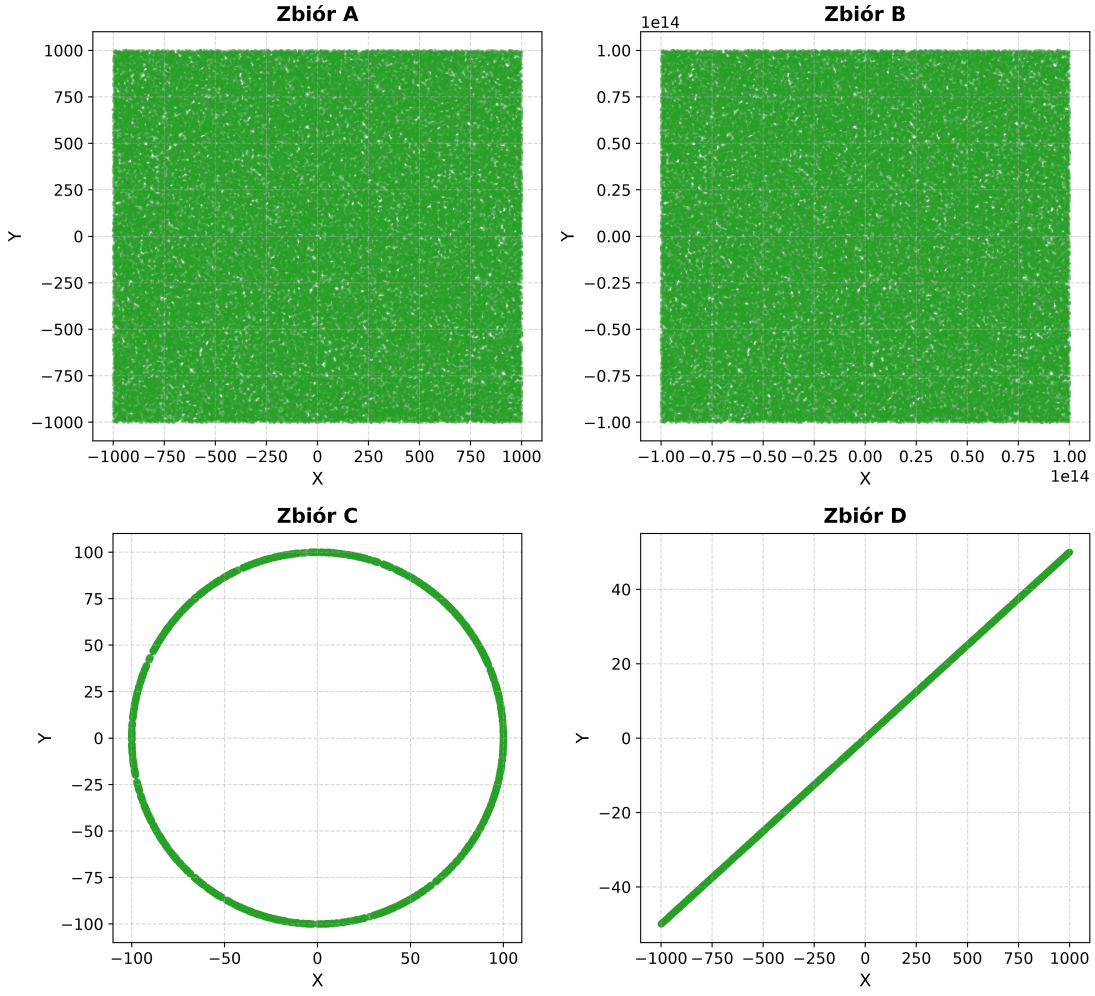
- 4 metody obliczeniowe: `custom_3x3`, `custom_2x2`, `numpy_3x3`, `numpy_2x2`,
- 2 typy precyzji: `float64` oraz `float32`,
- 6 progów tolerancji: $\varepsilon \in \left\{0.0, \quad 10^{-14}, \quad 10^{-12}, \quad 10^{-10}, \quad 10^{-8}, \quad 10^{-6}\right\}$.

Łącznie wykonano 240 prób kwalifikacyjnych.

4. Wyniki i analiza eksperymentalna

4.1. Wizualizacja wygenerowanych zbiorów danych

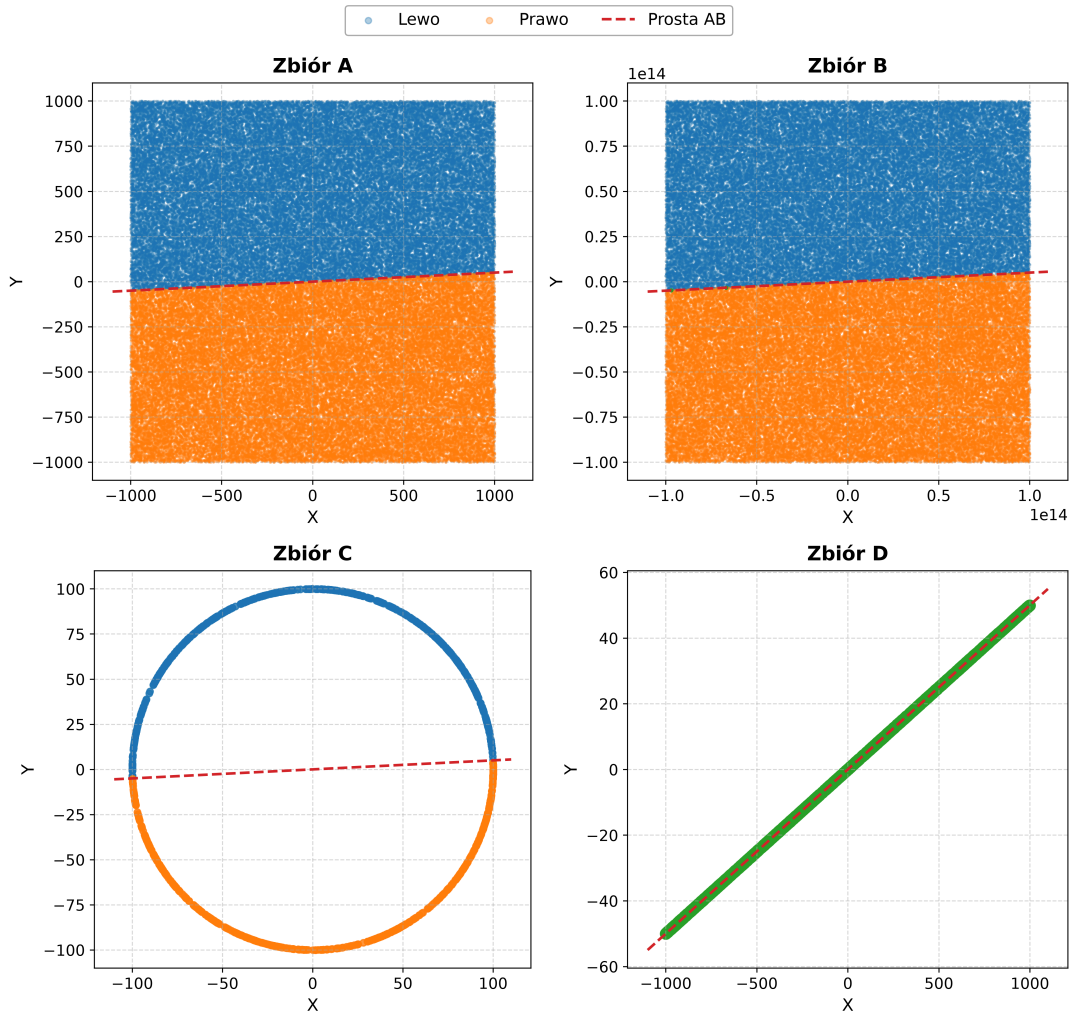
Na Rysunku 1 przedstawiono rozkład przestrzenny punktów w czterech podstawowych zbiorach przed wykonaniem klasyfikacji. Zbiory A i B reprezentują płaszczyzny losowe o zróżnicowanej skali współrzędnych, Zbiór C tworzy okrąg, natomiast Zbiór D skupia punkty współliniowe.



Rysunek 1: Wygenerowane zbiory testowe

4.2. Wizualizacja klasyfikacji punktów

Na Rysunku 2 zilustrowano podział punktów na kategorie orientacji (Lewo, Prawo, Na prostej) wyznaczony przez prostą AB dla metody `custom_2x2` z progiem $\varepsilon = 10^{-10}$.



Rysunek 2: Klasyfikacja punktów dla wszystkich zbiorów testowych

4.3. Wyniki dla zbiorów standardowych (Zbiór A i Zbiór C)

W tabeli 1 zestawiono wyniki klasyfikacji uzyskane dla zbiorów A i C. Wyniki te były w 100% identyczne dla wszystkich czterech badanych metod, obu precyzji sprzętowych (`float64`, `float32`) oraz każdej badanej wartości tolerancji ε.

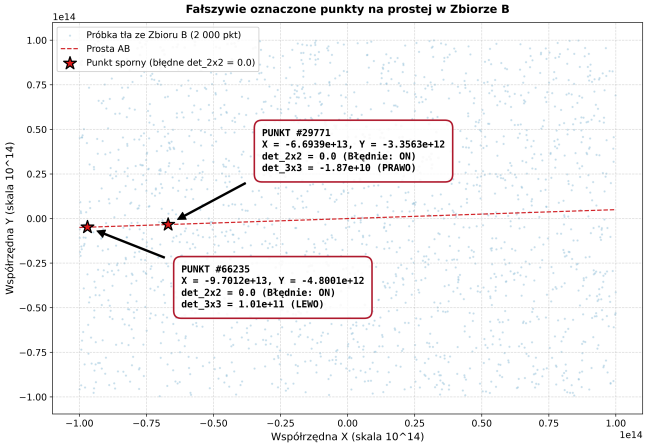
Zbiór	Rozmiar próby	Lewo	Prawo	Na prostej
Zbiór A	100000	50 098	49 902	0
Zbiór C	1000	509	491	0

Tabela 1: Stabilne wyniki klasyfikacji dla Zbioru A oraz Zbioru C.

Wnioski z analizy zbiorów standardowych: Dla skal ($[-1000, 1000]$), gdy punkty leżą w bezpiecznej odległości od prostej bazowej, wybór metody wyznacznika oraz precyzji nie wpływa na poprawność geometryczną.

4.4. Zbiór B: Fałszywie oznaczone punkty na prostej

W Zbiorze B (10^5 punktów o współrzędnych rzędu 10^{14}) pojawia się problem fałszywego uznawania punktów za leżące na prostej.



Rysunek 3: Fałszywie oznaczone punkty na prostej w Zbiorze B

Metoda	Precyzja	ε	Lewo	Prawo	Na prostej	Błąd [%]
custom_3x3	float64	0.0	50 098	49 902	0	0.00 %
custom_2x2	float64	0.0	50 097	49 901	**2**	0.002 %
numpy_3x3	float64	0.0	50 098	49 902	0	0.00 %
numpy_2x2	float64	0.0	50 098	49 902	0	0.00 %
custom_3x3	float32	0.0	50 098	49 902	0	0.00 %
custom_2x2	**float32**	0.0	**0**	**0**	**100 000**	**100.00 %**
numpy_3x3	float32	0.0	50 098	49 902	0	0.00 %
numpy_2x2	**float32**	0.0	6 615	6 723	**86 662**	**86.66 %**

Tabela 2: Zestawienie wyników klasyfikacji dla Zbioru B (10^5 punktów w skali 10^{14}).

Analiza anomalii w Zbiorze B:

1. **Całkowite załamanie formatu `float32`:** W formule 2×2 odejmowanie małych współrzędnych punktów bazowych od liczb rzędu 10^{14} sprawia, że małe wartości gubią się w zaokrągleniach 24-bitowej mantysy. W efekcie dla komputera obie różnice są identyczne, co daje wyznacznik 0.0 dla wszystkich 100 000 punktów (100% fałszywych współliniowości).
2. **Punkty sporne w precyzji `float64`:** Przy braku tolerancji ($\varepsilon = 0.0$) algorytmy dały sprzeczne wyniki dla dwóch punktów:
   - **Punkt nr 29771:** $X = -6.69 \times 10^{13}, Y = -3.36 \times 10^{12}$. Formuła `custom_2x2` zwróciła dokładnie 0.0 (błędnie: Na prostej), podczas gdy `custom_3x3` zwróciła wartość ujemną (Prawo).
   - **Punkt nr 66235:** $X = -9.70 \times 10^{13}, Y = -4.80 \times 10^{12}$. Formuła `custom_3x3` zwróciła wartość dodatnią (Lewo).

   Weryfikacja ręczna: Podstawienie współrzędnych do równania prostej $y = 0.05x + 0.05$ jednoznacznie wykazuje, że punkty te leżą daleko od prostej: Punkt 29771 leży pod prostą (rzeczywista strona: Prawo), natomiast Punkt 66235 leży nad prostą (rzeczywista strona: Lewo).

   Wpływ przeskalowania punktów prostej: Gdy punkty bazowe wyznaczymy na tej samej prostej, ale w skali odpowiadającej badanym danym ($A = [-10^{14}, -5 \cdot 10^{12} + 0.05], B = [10^{14}, 5 \cdot 10^{12} + 0.05]$), błąd fałszywej współliniowości całkowicie znika. Obie metody uzyskują zgodne orzeczenia odpowiadające prawdzie geometrycznej (Punkt 29771: Prawo, Punkt 66235: Lewo).

4.5. Zbiór D: Błędy klasyfikacji na punktach współliniowych (wersja kierunkowa)

W Zbiorze D każdy z 1000 punktów leży matematycznie dokładnie na prostej AB. Poniższa Tabela 3 przedstawia wyniki dla zbioru generowanego równaniem kierunkowym ($y = 0.05x + 0.05$). W idealnej arytmetyce prawidłowy wynik to wyłącznie etykieta „Na prostej”.

Część	Metoda	Precyzja	ε	Lewo	Prawo	Na prostej
(I)	custom_3x3	float64	0.0	178	405	**417**
(I)	custom_2x2	float64	0.0	156	156	**688**
(I)	numpy_3x3	float64	0.0	453	545	**2**
(I)	numpy_2x2	float64	0.0	503	497	**0**
(II)	custom_3x3	float64	10^{-14}	0	0	**1 000**
(II)	custom_2x2	float64	10^{-14}	147	147	**706**
(II)	numpy_3x3	float64	10^{-10}	0	0	**1 000**
(II)	numpy_2x2	float64	10^{-10}	0	0	**1 000**
(III)	custom_3x3	float32	0.0	298	315	**387**
(III)	custom_2x2	float32	0.0	163	175	**662**
(III)	numpy_3x3	float32	0.0	489	511	**0**
(III)	numpy_2x2	float32	0.0	513	487	**0**

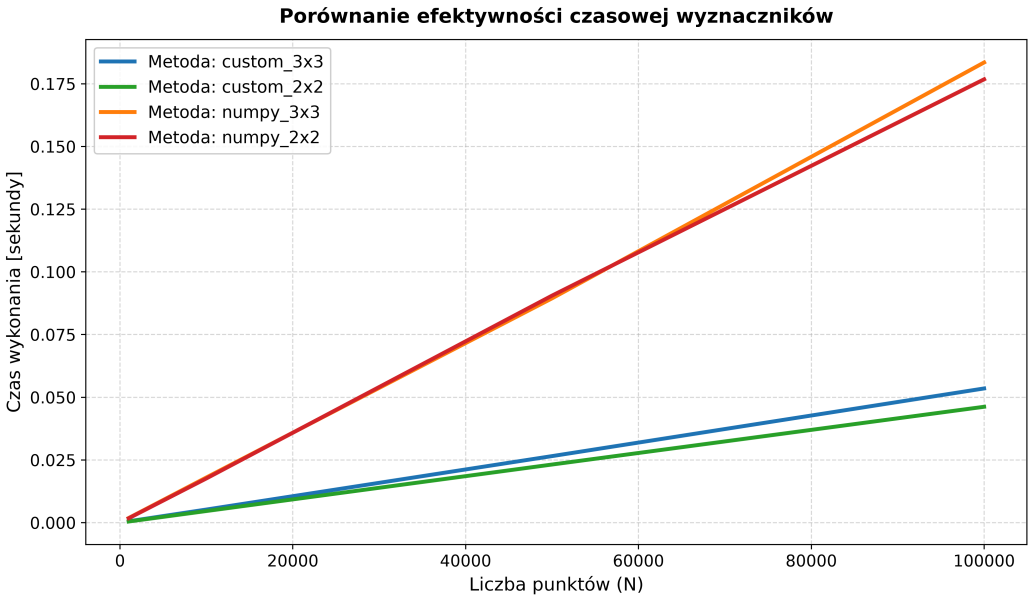
Tabela 3: Wyniki kategoryzacji punktów Zbioru D (równanie kierunkowe) z podziałem na warianty analityczne.

Szczegółowa analiza wyników Zbioru D (odniesienie do części tabeli):

- **(I) Brak tolerancji ($\varepsilon = 0.0$):** Komputer uznaje za współliniowe zaledwie część punktów: 688 w metodzie `custom_2x2`, 417 w `custom_3x3` oraz zaledwie 0–2 punktów w funkcjach biblioteki NumPy. Wynika to z faktu, że współczynnik prostej 0.05 jest ułamkiem nieskończonym w systemie dwójkowym, co wprowadza względne błędy zaokrąglenia współrzędnych rzędu 10^{-16}. W wyznaczniku dają one $|D|$ rzędu 10^{-15} (3×3) i 10^{-12}–10^{-11} (2×2). Porównanie wyników punkt po punkcie między `custom_3x3` a `custom_2x2` wykazuje aż **677 sprzecznych decyzji** (dla 677 punktów z 1000 algorytmy wskazały inną stronę, np. jedna metoda uznała punkt za współliniowy, a druga za leżący po lewej lub prawej stronie, co wynika z odmiennej kolejności działań arytmetycznych).
- **(II) Działanie progu tolerancji ($\varepsilon > 0$):** Wprowadzenie progu $\varepsilon = 10^{-10}$ całkowicie niweluje wpływ drobnych błędów zaokrągleń komputera, przywracając 100% poprawnych orzeczeń (dokładnie 1000 punktów na prostej) dla wszystkich badanych algorytmów w precyzji `float64`.
- **(III) Wpływ pojedynczej precyzji `float32`:** Format 32-bitowy dysponuje zaledwie 24 bitami mantysy (ok. 7 cyfr dziesiętnych). Dla współrzędnych z przedziału $[-1000, 1000]$ krok siatki jest zbyt duży. W bibliotece NumPy wyznacznik nigdy nie osiąga idealnego zera — bez tolerancji zbiór zostaje podzielony niemal po połowie na lewo i prawo (489/511 i 513/487), dając **0 punktów na prostej**. Metody autorskie `custom_2x2` (662) oraz `custom_3x3` (387) wykazują częściowe orzeczenia zerowe, gdyż odejmowanie częściej daje identyczne bity w rejestrach. W `float32` żaden badany próg ($\varepsilon \leq 10^{-6}$) nie daje pełnej poprawności (najlepiej: 670 punktów, `custom_2x2`). Maksymalne $|D|$ to ok. $7.6 \cdot 10^{-6}$ (3×3) i $3.9 \cdot 10^{-3}$ (2×2), więc potrzebne byłyby progi rzędu 10^{-5} i 10^{-2}.
- **(IV) Wpływ postaci równania (wersja kierunkowa vs parametryczna):** Postać równania wpływa na błędy numeryczne przy braku tolerancji ($\varepsilon = 0.0$): w równaniu kierunkowym operacja $y = 0.05x + 0.05$ wprowadza błąd zaokrąglenia już podczas generowania współrzędnych ($0.05 = \frac{1}{20}$ to ułamek okresowy w systemie dwójkowym). W wersji parametrycznej ($x = -1 + 2t, y = 0.1t$) mnożenie przez 2 jest dokładne binarnie, a y nie wymaga dodawania wyrazu wolnego. W rezultacie przy $\varepsilon = 0.0$ metoda `custom_3x3` zakwalifikowała 417 punktów w wersji kierunkowej wobec 325 w parametrycznej, a `custom_2x2` — 688 wobec 678. Po włączeniu bezpiecznego progu ($\varepsilon \geq 10^{-10}$) oba warianty dają jednak identyczny wynik: 1000 punktów na prostej (100%).

4.6. Porównanie efektywności czasowej (Benchmark)

Przeprowadzono pomiar czasu klasyfikacji za pomocą `time.perf_counter()` dla rozmiarów próby $N \in \{1\,000, 10\,000, 50\,000, 100\,000\}$.



Rysunek 4: Zależność czasu wykonania od rozmiaru próby N dla czterech badanych metod.

Rozmiar próby N	custom_2x2	custom_3x3	numpy_2x2	numpy_3x3
1 000	0.49 ms	0.56 ms	1.83 ms	1.91 ms
10 000	4.84 ms	5.65 ms	18.19 ms	18.84 ms
50 000	23.91 ms	26.96 ms	88.12 ms	91.88 ms
100 000	48.52 ms	55.41 ms	179.15 ms	186.06 ms

Tabela 4: Średnie czasy wykonania klasyfikacji na procesorze Apple M4.

Wnioski z analizy wydajnościowej:

1. Implementacje autorskie (`custom_*`) okazały się **3.4–3.8 raza szybsze** od wywołań biblioteki NumPy w pętli. Dla 100 000 punktów `custom_2x2` wykonuje się w 48.5 ms, podczas gdy `numpy_3x3` wymaga 186.1 ms.
2. Narzut NumPy wynika z wywoływania funkcji w pętli `for` interpretera Pythona — koszt alokacji obiektów macierzy przewyższa czas samej dekompozycji.
3. Wszystkie metody wykazują stabilną złożoność liniową $O(N)$.

5. Wnioski końcowe

Na podstawie przeprowadzonych eksperymentów sformułowano następujące wnioski:

1. **Niewystarczalność formatu `float32`:** Pojedyncza precyzja (ok. 7 cyfr znaczących) nie nadaje się do obliczeń geometrycznych przy dużych współrzędnych ($\geq 10^7$). Dochodzi wtedy do utraty cyfr znaczących przy odejmowaniu dużych liczb od siebie — małe współrzędne prostej są całkowicie „pochłaniane" przez duże liczby punktu, co skutkuje masowym orzekaniem fałszywej współliniowości (aż 100% błędu w Zbiorze B).

2. **Porównanie formuły różnicowej 2×2 z formułą pełną 3×3:** W Zbiorze D (współrzędne od -1000 do 1000) wzór 2×2 częściej daje wynik dokładnie równy zeru (688 punktów na prostej bez tolerancji wobec 417 dla metody 3×3), jednak jego pozostałe błędy są znacznie większe: maksymalne $|D|$ to ok. $7 \cdot 10^{-12}$ wobec ok. $7 \cdot 10^{-15}$ dla 3×3. Dlatego `custom_3x3` osiąga pełną poprawność już przy $\varepsilon = 10^{-14}$, a `custom_2x2` dopiero przy $\varepsilon = 10^{-10}$. Przyczyną jest to, że punkty bazowe A, B leżą blisko początku układu, więc w formule 3×3 mnożone są małe liczby, a w formule 2×2 — różnice rzędu wielkości punktu C. Jednak w przypadku skrajnych liczb (10^{14}), odejmowanie to potrafi zgubić nachylenie prostej, jeśli wektor bazowy nie zostanie odpowiednio dopasowany do rzędu wielkości punktów.

3. **Konieczność stosowania progu tolerancji ε:** Sprawdzanie idealnego zera ($D == 0.0$) w komputerze prowadzi w praktyce do błędnych wniosków. Dla Zbioru D brak tolerancji odrzucił od 31% do aż 100% punktów faktycznie leżących na prostej z powodu błędów zaokrągleń (rzędu 10^{-15}–10^{-11} w wartości wyznacznika). Dobór bezpiecznego progu $\varepsilon = 10^{-10}$ przywrócił pełną 100% poprawność klasyfikacji we wszystkich badanych metodach w precyzji `float64`; w `float32` żaden badany próg nie był wystarczający.

4. **Zależność tolerancji od rzędu wielkości danych:** Próg tolerancji ε nie może być jednakowy dla każdego zadania — powinien być dobierany w zależności od wielkości współrzędnych (dla liczb w zakresie do 1000 wystarcza $\varepsilon = 10^{-10}$, natomiast przy liczbach rzędu 10^{14} operacje wymagają zupełnie innej skali).

5. **Szybkość obliczeń:** Własna, prosta funkcja licząca wyznacznik ze wzoru arytmetycznego okazała się niemal 4 razy szybsza niż gotowa funkcja `np.linalg.det` wywoływana w pętli. Wynika to z faktu, że wywoływanie biblioteki w pętli dla każdego punktu z osobna wiąże się z dużym narzutem czasowym Pythona. Nie świadczy to o powolności samej biblioteki NumPy — obliczenie wzoru 2×2 w postaci wektorowej, na całych tablicach naraz, byłoby szybsze od obu wariantów z pętlą.